



It's our 10 Year Anniversary Sale! Celebrate today with 69% off. Claim Discount



Log In

Join for free

Home > Courses > Grokking the Coding Interview Patterns

Popular ★★★★★ 4.5 📊 Intermediate ⌚ 85h ↻ Updated this week

Grokking the Coding Interview Patterns 📖

Master 24 essential coding patterns to solve thousands of LeetCode-style questions. Efficiently prepare for coding interviews with the ultimate course created by FAANG engineers.

Start Course

Join 2.7 million developers at     

This course includes

✦ Personalized Learning, Powered by AI

 749 Lessons  11 Mock Interviews  391 Quizzes  390 Challenges  Certificate of Completion

Pick a Language



C#



C++



Go



Java



JavaScript



Python

 Overview Content Reviews Related

With thousands of potential questions to account for, preparing for the coding interview can feel like an impossible challenge. Yet with a strategic approach, coding interview prep doesn't have to take more than a few weeks. Stop drilling endless sets of practice problems, and prepare more efficiently by learning coding interview patterns.

This course teaches you the underlying patterns behind common coding interview questions. By learning these essential patterns, you will be able to unpack and answer any problem the right way — just by assessing the problem statement. This approach was created by FAANG hiring managers to help you prepare for the typical rounds of interviews at major tech companies like Apple, Google, Meta, Microsoft, and Amazon.

Before long, you will have the skills you need to unlock even the most challenging questions, grok the coding interview, and level up your career with confidence.

This course is also available in JavaScript, Python, Go, and C++ — with more coming soon!

WHAT YOU'LL LEARN

- ✓ Understand the essential patterns behind common coding interview questions — without having to drill endless problem sets
- ✓ Identify the underlying pattern of each interview question by assessing the problem statement (and learn the tricks required to solve it)
- ✓ Practice your skills in a hands-on, setup-free coding environment
- ✓ Learn to efficiently evaluate the tradeoffs between time and space complexity in different solutions
- ✓ Develop a flexible conceptual framework for solving any question, by connecting problem characteristics, solution techniques, and coding design patterns

TAKEAWAY SKILLS

Java

Programming Language

Prepare for Interview

Coding Interview Patterns

Data Structures

Python

Content

749 Lessons • 391 Quizzes • 390 Challenges

Collapse All ⬆

1. Getting Started ^

Get familiar with coding design patterns to excel in coding interviews.

- ☐ [Course Overview](#)
- ☐ [Who Should Take This Course](#)

2. Two Pointers ^

Walk through the fundamentals of the two-pointers technique for solving various coding problems.

- ☐ [Introduction to Two Pointers](#)
- ☐ [Valid Palindrome](#)
- ☐ [Solution: Valid Palindrome](#)
- ☐ [3Sum](#)
- ☐ [Solution: 3Sum](#)
- ☐ [Remove Nth Node from End of List](#)

- ☐ [Solution: Remove Nth Node from End of List](#)
- ☐ [Sort Colors](#)
- ☐ [Solution: Sort Colors](#)
- ☐ [Reverse Words in a String](#)
- ☐ [Solution: Reverse Words in a String](#)
- ☐ [Valid Word Abbreviation](#)
- ☐ [Solution: Valid Word Abbreviation](#)
- ☐ [Strobogrammatic Number](#)
- ☐ [Solution: Strobogrammatic Number](#)
- ☐ [Minimum Number of Moves to Make Palindrome](#)
- ☐ [Solution: Minimum Number of Moves to Make Palindrome](#)
- ☐ [Next Palindrome Using Same Digits](#)
- ☐ [Solution: Next Palindrome Using Same Digits](#)
- ☐ [Lowest Common Ancestor of a Binary Tree III](#)
- ☐ [Solution: Lowest Common Ancestor of a Binary Tree III](#)
- ☐ [Count Pairs Whose Sum is Less than Target](#)
- ☐ [Solution: Count Pairs Whose Sum is Less than Target](#)
- ☐ [Valid Palindrome II](#)



Two Pointers

Mock Interview

Premium ⓘ

3. Fast and Slow Pointers



Go hands-on with fast and slow pointers to detect cycles, middle nodes, and duplicates.

- ☐ [Introduction to Fast and Slow Pointers](#)
- ☐ [Happy Number](#)
- ☐ [Solution: Happy Number](#)
- ☐ [Linked List Cycle](#)
- ☐ [Solution: Linked List Cycle](#)

- ☐ [Middle of the Linked List](#)
- ☐ [Solution: Middle of the Linked List](#)
- ☐ [Circular Array Loop](#)
- ☐ [Solution: Circular Array Loop](#)
- ☐ [Find the Duplicate Number](#)
- ☐ [Solution: Find the Duplicate Number](#)
- ☐ [Palindrome Linked List](#)
- ☐ [Solution: Palindrome Linked List](#)
- ☐ [Maximum Twin Sum of a Linked List](#)
- ☐ [Solution: Maximum Twin Sum of a Linked List](#)
- ☐ [Split a Circular Linked List](#)
- ☐ [Solution: Split a Circular Linked List](#)
- ☐ [Linked List Cycle III](#)
- ☐ [Solution: Linked List Cycle III](#)
- ☐ [Linked List Cycle IV](#)
- ☐ [Solution: Linked List Cycle IV](#)



Fast and Slow Pointers

Mock Interview

Premium ⓘ

4. Sliding Window ^

Grasp the fundamentals of efficiently solving subarray and substring problems using the sliding window technique.

- ☐ [Introduction to Sliding Window](#)
- ☐ [Repeated DNA Sequences](#)
- ☐ [Solution: Repeated DNA Sequences](#)
- ☐ [Sliding Window Maximum](#)
- ☐ [Solution: Sliding Window Maximum](#)
- ☐ [Minimum Window Subsequence](#)
- ☐ [Solution: Minimum Window Subsequence](#)

- ☐ [Longest Repeating Character Replacement](#)
- ☐ [Solution: Longest Repeating Character Replacement](#)
- ☐ [Minimum Window Substring](#)
- ☐ [Solution: Minimum Window Substring](#)
- ☐ [Longest Substring without Repeating Characters](#)
- ☐ [Solution: Longest Substring without Repeating Characters](#)
- ☐ [Minimum Size Subarray Sum](#)
- ☐ [Solution: Minimum Size Subarray Sum](#)
- ☐ [Maximum Average Subarray I](#)
- ☐ [Solution: Maximum Average Subarray I](#)
- ☐ [Diet Plan Performance](#)
- ☐ [Solution: Diet Plan Performance](#)
- ☐ [Fruit Into Baskets](#)
- ☐ [Solution: Fruit Into Baskets](#)
- ☐ [Contains Duplicate II](#)
- ☐ [Solution: Contains Duplicate II](#)
- ☐ [Frequency of the Most Frequent Element](#)
- ☐ [Solution: Frequency of the Most Frequent Element](#)
- ☐ [Subarrays with K Different Integers](#)
- ☐ [Solution: Subarrays with K Different Integers](#)



Sliding Window

Mock Interview

Premium ⓘ

5. Merge Intervals



Map out the steps for solving overlapping interval problems in various contexts.

- ☐ [Introduction to Merge Intervals](#)
- ☐ [Merge Intervals](#)
- ☐ [Solution: Merge Intervals](#)

- ☐ [Insert Interval](#)
- ☐ [Solution: Insert Interval](#)
- ☐ [Interval List Intersections](#)
- ☐ [Solution: Interval List Intersections](#)
- ☐ [Employee Free Time](#)
- ☐ [Solution: Employee Free Time](#)
- ☐ [Count Days Without Meetings](#)
- ☐ [Solution: Count Days Without Meetings](#)
- ☐ [Remove Covered Intervals](#)
- ☐ [Solution: Remove Covered Intervals](#)
- ☐ [Meeting Rooms II](#)

6. In-Place Manipulation of a Linked List ^

Follow efficient in-place linked list manipulation techniques for various node reordering and removal tasks.

- ☐ [Introduction to In-Place Manipulation of a Linked List](#)
- ☐ [Reverse Linked List](#)
- ☐ [Solution: Reverse Linked List](#)
- ☐ [Reverse Nodes in k-Group](#)
- ☐ [Solution: Reverse Nodes in k-Group](#)
- ☐ [Reverse Linked List II](#)
- ☐ [Solution: Reverse Linked List II](#)
- ☐ [Reorder List](#)
- ☐ [Solution: Reorder List](#)
- ☐ [Swapping Nodes in a Linked List](#)
- ☐ [Solution: Swapping Nodes in a Linked List](#)
- ☐ [Reverse Nodes in Even Length Groups](#)
- ☐ [Solution: Reverse Nodes in Even Length Groups](#)
- ☐ [Remove Duplicates from Sorted List](#)

- ☐ [Solution: Remove Duplicates from Sorted List](#)
- ☐ [Remove Linked List Elements](#)
- ☐ [Solution: Remove Linked List Elements](#)
- ☐ [Split Linked List in Parts](#)
- ☐ [Solution: Split Linked List in Parts](#)
- ☐ [Delete N Nodes After M Nodes of a Linked List](#)
- ☐ [Solution: Delete N Nodes After M Nodes of a Linked List](#)
- ☐ [Insert into a Sorted Circular Linked List](#)
- ☐ [Solution: Insert into a Sorted Circular Linked List](#)
- ☐ [Swap Nodes in Pairs](#)

7. Heaps ^

Build on efficient data structures and algorithms using heaps for dynamic data processing.

- ☐ [Introduction to Heaps](#)
- ☐ [IPO](#)
- ☐ [Solution: IPO](#)
- ☐ [Find Median from Data Stream](#)
- ☐ [Solution: Find Median from Data Stream](#)
- ☐ [Sliding Window Median](#)
- ☐ [Solution: Sliding Window Median](#)
- ☐ [Schedule Tasks on Minimum Machines](#)
- ☐ [Solution: Schedule Tasks on Minimum Machines](#)
- ☐ [Meeting Rooms III](#)
- ☐ [Solution: Meeting Rooms III](#)
- ☐ [Largest Number After Digit Swaps by Parity](#)
- ☐ [Solution: Largest Number After Digit Swaps by Parity](#)
- ☐ [Find Right Interval](#)
- ☐ [Solution: Find Right Interval](#)

- ☐ [Minimum Cost to Connect Sticks](#)
- ☐ [Solution: Minimum Cost to Connect Sticks](#)
- ☐ [Longest Happy String](#)
- ☐ [Solution: Longest Happy String](#)
- ☐ [Maximum Average Pass Ratio](#)
- ☐ [Solution: Maximum Average Pass Ratio](#)
- ☐ [The Number of the Smallest Unoccupied Chair](#)
- ☐ [Solution: The Number of the Smallest Unoccupied Chair](#)



Heaps

Mock Interview

Premium ⓘ

8. K-way merge



Step through the key techniques and algorithms for efficiently merging sorted data structures.

- ☐ [Introduction to K-way Merge](#)
- ☐ [Merge Sorted Array](#)
- ☐ [Solution: Merge Sorted Array](#)
- ☐ [Kth Smallest Number in M Sorted Lists](#)
- ☐ [Solution: Kth Smallest Number in M Sorted Lists](#)
- ☐ [Find K Pairs with Smallest Sums](#)
- ☐ [Solution: Find K Pairs with Smallest Sums](#)
- ☐ [Merge K Sorted Lists](#)
- ☐ [Solution: Merge K Sorted Lists](#)
- ☐ [Kth Smallest Element in a Sorted Matrix](#)
- ☐ [Solution: Kth Smallest Element in a Sorted Matrix](#)
- ☐ [Kth Smallest Prime Fraction](#)
- ☐ [Solution: Kth Smallest Prime Fraction](#)
- ☐ [Super Ugly Number](#)

☐ [Solution: Super Ugly Number](#)

9. Top K Elements ^

Unpack the core of identifying top K elements using heaps in various scenarios.

☐ [Introduction to Top K Elements](#)

☐ [Kth Largest Element in a Stream](#)

☐ [Solution: Kth Largest Element in a Stream](#)

☐ [Reorganize String](#)

☐ [Solution: Reorganize String](#)

☐ [K Closest Points to Origin](#)

☐ [Solution: K Closest Points to Origin](#)

☐ [Top K Frequent Elements](#)

☐ [Solution: Top K Frequent Elements](#)

☐ [Kth Largest Element in an Array](#)

☐ [Solution: Kth Largest Element in an Array](#)

☐ [Third Maximum Number](#)

☐ [Solution: Third Maximum Number](#)

☐ [Find Subsequence of Length K with the Largest Sum](#)

☐ [Solution: Find Subsequence of Length K with the Largest Sum](#)

☐ [Minimum Cost to Hire K Workers](#)

☐ [Solution: Minimum Cost to Hire K Workers](#)

☐ [Maximal Score After Applying K Operations](#)

☐ [Solution: Maximal Score After Applying K Operations](#)

☐ [Find the Kth Largest Integer in the Array](#)

☐ [Solution: Find the Kth Largest Integer in the Array](#)

☐ [Maximum Performance of a Team](#)

☐ [Solution: Maximum Performance of a Team](#)

☐ [Smallest Range Covering Elements from K Lists](#)

☐ [Solution: Smallest Range Covering Elements from K Lists](#)

- ☐ [K Maximum Sum Combinations From Two Arrays](#)
 - ☐ [Solution: K Maximum Sum Combinations From Two Arrays](#)
 - ☐ [K Empty Slots](#)
 - ☐ [Solution: K Empty Slots](#)
-

10. Modified Binary Search ^

Break apart various search techniques using modified binary search for different problem scenarios.

- ☐ [Introduction to Modified Binary Search](#)
- ☐ [Binary Search](#)
- ☐ [Solution: Binary Search](#)
- ☐ [Search in Rotated Sorted Array](#)
- ☐ [Solution: Search in Rotated Sorted Array](#)
- ☐ [First Bad Version](#)
- ☐ [Solution: First Bad Version](#)
- ☐ [Random Pick with Weight](#)
- ☐ [Solution: Random Pick with Weight](#)
- ☐ [Find K Closest Elements](#)
- ☐ [Solution: Find K Closest Elements](#)
- ☐ [Single Element in a Sorted Array](#)
- ☐ [Solution: Single Element in a Sorted Array](#)
- ☐ [Maximum Value at a Given Index in a Bounded Array](#)
- ☐ [Solution: Maximum Value at a Given Index in a Bounded Array](#)
- ☐ [The K Weakest Rows in a Matrix](#)
- ☐ [Solution: The K Weakest Rows in a Matrix](#)
- ☐ [Split Array Largest Sum](#)
- ☐ [Solution: Split Array Largest Sum](#)
- ☐ [Find Minimum in Rotated Sorted Array II](#)
- ☐ [Solution: Find Minimum in Rotated Sorted Array II](#)

- ☐ [Maximum Running Time of N Computers](#)
- ☐ [Solution: Maximum Running Time of N Computers](#)
- ☐ [Minimize Max Distance to Gas Station](#)
- ☐ [Solution: Minimize Max Distance to Gas Station](#)
- ☐ [Divide Chocolate](#)
- ☐ [Solution: Divide Chocolate](#)
- ☐ [Search in Rotated Sorted Array II](#)



Modified Binary Search

Mock Interview

Premium ⓘ

11. Subsets



Grasp the fundamentals of subsets, permutations, letter combinations, balanced parentheses, and k-sum subsets.

- ☐ [Introduction to Subsets](#)
- ☐ [Subsets](#)
- ☐ [Solution: Subsets](#)
- ☐ [Permutations](#)
- ☐ [Solution: Permutations](#)
- ☐ [Letter Combinations of a Phone Number](#)
- ☐ [Solution: Letter Combinations of a Phone Number](#)
- ☐ [Generate Parentheses](#)
- ☐ [Solution: Generate Parentheses](#)
- ☐ [Letter Case Permutation](#)
- ☐ [Solution: Letter Case Permutation](#)
- ☐ [Letter Tile Possibilities](#)
- ☐ [Solution: Letter Tile Possibilities](#)
- ☐ [Find K-Sum Subsets](#)

12. Greedy Techniques



Dig deeper into the application and optimization of greedy algorithms across various problem scenarios.

- ☐ [Introduction to Greedy Techniques](#)
- ☐ [Jump Game](#)
- ☐ [Solution: Jump Game](#)
- ☐ [Boats to Save People](#)
- ☐ [Solution: Boats to Save People](#)
- ☐ [Gas Station](#)
- ☐ [Solution: Gas Station](#)
- ☐ [Two City Scheduling](#)
- ☐ [Solution: Two City Scheduling](#)
- ☐ [Minimum Number of Refueling Stops](#)
- ☐ [Solution: Minimum Number of Refueling Stops](#)
- ☐ [Largest Palindromic Number](#)
- ☐ [Solution: Largest Palindromic Number](#)
- ☐ [Assign Cookies](#)
- ☐ [Solution: Assign Cookies](#)
- ☐ [Rearranging Fruits](#)
- ☐ [Solution: Rearranging Fruits](#)
- ☐ [Number of Steps to Reduce a Binary Number to One](#)
- ☐ [Solution: Number of Steps to Reduce a Binary Number to One](#)
- ☐ [Maximum Swap](#)
- ☐ [Solution: Maximum Swap](#)
- ☐ [Can Place Flowers](#)
- ☐ [Solution: Can Place Flowers](#)
- ☐ [Largest Odd Number in String](#)
- ☐ [Solution: Largest Odd Number in String](#)
- ☐ [Candy](#)
- ☐ [Solution: Candy](#)

- ☐ [Minimum Replacements to Sort the Array](#)
- ☐ [Solution: Minimum Replacements to Sort the Array](#)
- ☐ [Jump Game II](#)
- ☐ [Best Time to Buy and Sell Stock](#)

13. Backtracking ^

See how it works to solve problems using backtracking, from N-Queens to Sudoku.

- ☐ [Introduction to Backtracking](#)
- ☐ [N-Queens II](#)
- ☐ [Solution: N-Queens II](#)
- ☐ [Word Search](#)
- ☐ [Solution: Word Search](#)
- ☐ [House Robber III](#)
- ☐ [Solution: House Robber III](#)
- ☐ [Restore IP Addresses](#)
- ☐ [Solution: Restore IP Addresses](#)
- ☐ [Flood Fill](#)
- ☐ [Solution: Flood Fill](#)
- ☐ [Minimum Moves to Spread Stones Over Grid](#)
- ☐ [Solution: Minimum Moves to Spread Stones Over Grid](#)
- ☐ [Binary Tree Paths](#)
- ☐ [Solution: Binary Tree Paths](#)
- ☐ [Binary Watch](#)
- ☐ [Solution: Binary Watch](#)
- ☐ [Optimal Account Balancing](#)
- ☐ [Solution: Optimal Account Balancing](#)
- ☐ [Split a String Into the Max Number of Unique Substrings](#)
- ☐ [Solution: Split a String Into the Max Number of Unique Substrings](#)
- ☐ [All Paths From Source to Target](#)

- ☐ [Solution: All Paths From Source to Target](#)
- ☐ [Sudoku Solver](#)
- ☐ [Matchsticks to Square](#)

14. Dynamic Programming ^

Master dynamic programming concepts, optimize solutions using memoization and tabulation for complex problems.

- ☐ [Introduction to Dynamic Programming](#)
- ☐ [0/1 Knapsack](#)
- ☐ [Solution: 0/1 Knapsack](#)
- ☐ [Coin Change](#)
- ☐ [Solution: Coin Change](#)
- ☐ [N-th Tribonacci Number](#)
- ☐ [Solution: N-th Tribonacci Number](#)
- ☐ [Partition Equal Subset Sum](#)
- ☐ [Solution: Partition Equal Subset Sum](#)
- ☐ [Counting Bits](#)
- ☐ [Solution: Counting Bits](#)
- ☐ [01 Matrix](#)
- ☐ [Solution: 01 Matrix](#)
- ☐ [House Robber II](#)
- ☐ [Solution: House Robber II](#)
- ☐ [Maximum Product Subarray](#)
- ☐ [Solution: Maximum Product Subarray](#)
- ☐ [Combination Sum](#)
- ☐ [Solution: Combination Sum](#)
- ☐ [Word Break](#)
- ☐ [Solution: Word Break](#)
- ☐ [Palindromic Substrings](#)

- ☐ [Solution: Palindromic Substrings](#)
- ☐ [Longest Common Subsequence](#)
- ☐ [Solution: Longest Common Subsequence](#)
- ☐ [Word Break II](#)
- ☐ [Solution: Word Break II](#)
- ☐ [Decode Ways](#)
- ☐ [Solution: Decode Ways](#)
- ☐ [Count the Number of Good Subsequences](#)
- ☐ [Solution: Count the Number of Good Subsequences](#)
- ☐ [Binary Tree Cameras](#)
- ☐ [Solution: Binary Tree Cameras](#)
- ☐ [Number of Ways to Form Target String Given a Dictionary](#)
- ☐ [Solution: Number of Ways to Form Target String Given a Dictionary](#)
- ☐ [Climbing Stairs](#)



Dynamic Programming

Mock Interview

Premium ⓘ

15. Cyclic Sort



Break down cyclic sort for efficient in-place sorting and finding missing, duplicate integers.

- ☐ [Introduction to Cyclic Sort](#)
- ☐ [Cyclic Sort](#)
- ☐ [Solution: Cyclic Sort](#)
- ☐ [Missing Number](#)
- ☐ [Solution: Missing Number](#)
- ☐ [First Missing Positive](#)
- ☐ [Solution: First Missing Positive](#)
- ☐ [Find the Corrupt Pair](#)
- ☐ [Solution: Find the Corrupt Pair](#)

- ☐ [Sort Array By Parity II](#)
- ☐ [Solution: Sort Array By Parity II](#)
- ☐ [Find the First K Missing Positive Numbers](#)

16. Topological Sort ^

Discover the logic behind topological sorting to solve dependency-based challenges and tasks.

- ☐ [Introduction to Topological Sort](#)
- ☐ [Compilation Order](#)
- ☐ [Solution: Compilation Order](#)
- ☐ [Alien Dictionary](#)
- ☐ [Solution: Alien Dictionary](#)
- ☐ [Verifying an Alien Dictionary](#)
- ☐ [Solution: Verifying an Alien Dictionary](#)
- ☐ [Course Schedule II](#)
- ☐ [Solution: Course Schedule II](#)
- ☐ [Course Schedule](#)
- ☐ [Solution: Course Schedule](#)
- ☐ [Build a Matrix with Conditions](#)
- ☐ [Solution: Build a Matrix with Conditions](#)
- ☐ [Longest Path With Different Adjacent Characters](#)
- ☐ [Solution: Longest Path With Different Adjacent Characters](#)
- ☐ [Find All Possible Recipes from Given Supplies](#)



Topological Sort

Mock Interview

Premium ⓘ



17. Sort and Search

Solve problems by organizing data through sorting and applying binary search, two-pointer, and sliding window for efficient solutions.

- ☐ [Introduction to Sort and Search](#)
- ☐ [Find the Distance Value Between Two Arrays](#)
- ☐ [Solution: Find the Distance Value Between Two Arrays](#)
- ☐ [Longest Subsequence With Limited Sum](#)
- ☐ [Solution: Longest Subsequence With Limited Sum](#)
- ☐ [Find Target Indices After Sorting Array](#)
- ☐ [Solution: Find Target Indices After Sorting Array](#)
- ☐ [Count Pairs in Two Arrays](#)
- ☐ [Solution: Count Pairs in Two Arrays](#)
- ☐ [Valid Triangle Number](#)
- ☐ [Solution: Valid Triangle Number](#)
- ☐ [Minimum Operations to Make All Array Elements Equal](#)
- ☐ [Solution: Minimum Operations to Make All Array Elements Equal](#)
- ☐ [Sum of Mutated Array Closest to Target](#)
- ☐ [Solution: Sum of Mutated Array Closest to Target](#)
- ☐ [Range Sum of Sorted Subarray Sums](#)
- ☐ [Solution: Range Sum of Sorted Subarray Sums](#)
- ☐ [Magnetic Force Between Two Balls](#)
- ☐ [Solution: Magnetic Force Between Two Balls](#)
- ☐ [Find K-th Smallest Pair Distance](#)
- ☐ [Solution: Find K-th Smallest Pair Distance](#)
- ☐ [Minimum Space Wasted from Packaging](#)
- ☐ [Solution: Minimum Space Wasted from Packaging](#)
- ☐ [Russian Doll Envelopes](#)
- ☐ [Solution: Russian Doll Envelopes](#)
- ☐ [Put Marbles in Bags](#)

- ☐ [Solution: Put Marbles in Bags](#)
- ☐ [H-Index](#)
- ☐ [Solution: H-Index](#)
- ☐ [Two Sum Less Than K](#)
- ☐ [Maximum Number of Integers to Choose from a Range I](#)

18. Matrices ^

Examine matrix operations, problem-solving methods, real-world applications, and algorithmic solutions for matrix traversal and transformations.

- ☐ [Introduction to Matrices](#)
- ☐ [Set Matrix Zeroes](#)
- ☐ [Solution: Set Matrix Zeroes](#)
- ☐ [Rotate Image](#)
- ☐ [Solution: Rotate Image](#)
- ☐ [Spiral Matrix](#)
- ☐ [Solution: Spiral Matrix](#)
- ☐ [Where Will the Ball Fall](#)
- ☐ [Solution: Where Will the Ball Fall](#)
- ☐ [Transpose Matrix](#)
- ☐ [Solution: Transpose Matrix](#)
- ☐ [Count Negative Numbers in a Sorted Matrix](#)
- ☐ [Solution: Count Negative Numbers in a Sorted Matrix](#)
- ☐ [Minimum Time Takes to Reach Destination Without Drowning](#)
- ☐ [Solution: Minimum Time Takes to Reach Destination Without Drowning](#)
- ☐ [Smallest Rectangle Enclosing Black Pixels](#)
- ☐ [Solution: Smallest Rectangle Enclosing Black Pixels](#)
- ☐ [Island Perimeter](#)
- ☐ [Solution: Island Perimeter](#)
- ☐ [Convert 1D Array Into 2D Array](#)

- ☐ [Solution: Convert 1D Array Into 2D Array](#)
- ☐ [Spiral Matrix II](#)
- ☐ [Solution: Spiral Matrix II](#)
- ☐ [Flip Columns For Maximum Number of Equal Rows](#)
- ☐ [Solution: Flip Columns For Maximum Number of Equal Rows](#)
- ☐ [Number of Spaces Cleaning Robot Cleaned](#)
- ☐ [Solution: Number of Spaces Cleaning Robot Cleaned](#)
- ☐ [Minimize Maximum Value in a Grid](#)
- ☐ [Solution: Minimize Maximum Value in a Grid](#)
- ☐ [Kth Smallest Number in Multiplication Table](#)
- ☐ [Solution: Kth Smallest Number in Multiplication Table](#)

19. Stacks ^

Grasp the fundamentals of stacks, their operations, and applications in various problems.

- ☐ [Introduction to Stacks](#)
- ☐ [Basic Calculator](#)
- ☐ [Solution: Basic Calculator](#)
- ☐ [Remove All Adjacent Duplicates In String](#)
- ☐ [Solution: Remove All Adjacent Duplicates In String](#)
- ☐ [Minimum Remove to Make Valid Parentheses](#)
- ☐ [Solution: Minimum Remove to Make Valid Parentheses](#)
- ☐ [Exclusive Time of Functions](#)
- ☐ [Solution: Exclusive Time of Functions](#)
- ☐ [Flatten Nested List Iterator](#)
- ☐ [Solution: Flatten Nested List Iterator](#)
- ☐ [Implement Queue Using Stacks](#)
- ☐ [Solution: Implement Queue Using Stacks](#)
- ☐ [Daily Temperatures](#)
- ☐ [Solution: Daily Temperatures](#)

- ☐ [Decode String](#)
- ☐ [Solution: Decode String](#)
- ☐ [Minimum String Length After Removing Substrings](#)
- ☐ [Solution: Minimum String Length After Removing Substrings](#)
- ☐ [Number of Valid Subarrays](#)
- ☐ [Solution: Number of Valid Subarrays](#)
- ☐ [Number of Visible People in a Queue](#)
- ☐ [Solution: Number of Visible People in a Queue](#)
- ☐ [Valid Parentheses](#)

20. Graphs



Take a closer look at graph theory, algorithms, traversals, and solving various graph-related problems.

- ☐ [Introduction to Graphs](#)
- ☐ [Network Delay Time](#)
- ☐ [Solution: Network Delay Time](#)
- ☐ [Paths in Maze That Lead to Same Room](#)
- ☐ [Solution: Paths in Maze That Lead to Same Room](#)
- ☐ [Clone Graph](#)
- ☐ [Solution: Clone Graph](#)
- ☐ [Graph Valid Tree](#)
- ☐ [Solution: Graph Valid Tree](#)
- ☐ [Bus Routes](#)
- ☐ [Solution: Bus Routes](#)
- ☐ [Reconstruct Itinerary](#)
- ☐ [Solution: Reconstruct Itinerary](#)
- ☐ [Find the Town Judge](#)
- ☐ [Solution: Find the Town Judge](#)
- ☐ [Find Center of Star Graph](#)

- ☐ [Solution: Find Center of Star Graph](#)
- ☐ [Lucky Numbers in a Matrix](#)
- ☐ [Solution: Lucky Numbers in a Matrix](#)
- ☐ [Path with Maximum Probability](#)
- ☐ [Solution: Path with Maximum Probability](#)
- ☐ [Tree Diameter](#)
- ☐ [Solution: Tree Diameter](#)
- ☐ [Reorder Routes to Make All Paths Lead to the City Zero](#)
- ☐ [Solution: Reorder Routes to Make All Paths Lead to the City Zero](#)
- ☐ [Minimum Cost to Make at Least One Valid Path in a Grid](#)
- ☐ [Solution: Minimum Cost to Make at Least One Valid Path in a Grid](#)
- ☐ [Longest Cycle in a Graph](#)
- ☐ [Solution: Longest Cycle in a Graph](#)
- ☐ [Shortest Cycle in a Graph](#)
- ☐ [Solution: Shortest Cycle in a Graph](#)



Graphs

Mock Interview

Premium ⓘ

21. Tree Depth-First Search



See how it works to explore, transform, analyze, and optimize various binary tree operations.

- ☐ [Introduction to Tree Depth-First Search](#)
- ☐ [Flatten Binary Tree to Linked List](#)
- ☐ [Solution: Flatten Binary Tree to Linked List](#)
- ☐ [Diameter of Binary Tree](#)
- ☐ [Solution: Diameter of Binary Tree](#)
- ☐ [Serialize and Deserialize Binary Tree](#)
- ☐ [Solution: Serialize and Deserialize Binary Tree](#)
- ☐ [Invert Binary Tree](#)

- ☐ [Solution: Invert Binary Tree](#)
- ☐ [Binary Tree Maximum Path Sum](#)
- ☐ [Solution: Binary Tree Maximum Path Sum](#)
- ☐ [Convert Sorted Array to Binary Search Tree](#)
- ☐ [Solution: Convert Sorted Array to Binary Search Tree](#)
- ☐ [Build Binary Tree from Preorder and Inorder Traversal](#)
- ☐ [Solution: Build Binary Tree from Preorder and Inorder Traversal](#)
- ☐ [Binary Tree Right Side View](#)
- ☐ [Solution: Binary Tree Right Side View](#)
- ☐ [Lowest Common Ancestor of a Binary Tree](#)
- ☐ [Solution: Lowest Common Ancestor of a Binary Tree](#)
- ☐ [Validate Binary Search Tree](#)
- ☐ [Solution: Validate Binary Search Tree](#)
- ☐ [Nested List Weight Sum II](#)
- ☐ [Solution: Nested List Weight Sum II](#)
- ☐ [Inorder Successor in BST](#)
- ☐ [Solution: Inorder Successor in BST](#)
- ☐ [Height of Binary Tree After Subtree Removal Queries](#)
- ☐ [Solution: Height of Binary Tree After Subtree Removal Queries](#)
- ☐ [Delete Nodes And Return Forest](#)
- ☐ [Solution: Delete Nodes And Return Forest](#)
- ☐ [Maximum Depth of Binary Tree](#)
- ☐ [Kth Smallest Element in a BST](#)

22. Tree Breadth-First Search



Build on performing efficient tree traversals with BFS approaches for various problems.

- ☐ [Introduction to Tree Breadth-First Search](#)
- ☐ [Binary Tree Level Order Traversal](#)
- ☐ [Solution: Binary Tree Level Order Traversal](#)

- ☐ [Binary Tree Zigzag Level Order Traversal](#)
- ☐ [Solution: Binary Tree Zigzag Level Order Traversal](#)
- ☐ [Populating Next Right Pointers in Each Node](#)
- ☐ [Solution: Populating Next Right Pointers in Each Node](#)
- ☐ [Vertical Order Traversal of a Binary Tree](#)
- ☐ [Solution: Vertical Order Traversal of a Binary Tree](#)
- ☐ [Symmetric Tree](#)
- ☐ [Solution: Symmetric Tree](#)
- ☐ [Word Ladder](#)
- ☐ [Solution: Word Ladder](#)
- ☐ [Two Sum IV – Input Is a BST](#)
- ☐ [Solution: Two Sum IV – Input Is a BST](#)
- ☐ [Find Minimum Diameter After Merging Two Trees](#)
- ☐ [Solution: Find Minimum Diameter After Merging Two Trees](#)
- ☐ [Connect All Siblings of a Binary Tree](#)



Tree Breadth First Search

Mock Interview

Premium ⓘ

23. Trie



Step through efficiently managing strings with trie data structures and practical applications.

- ☐ [Introduction to Trie](#)
- ☐ [Implement Trie \(Prefix Tree\)](#)
- ☐ [Solution: Implement Trie \(Prefix Tree\)](#)
- ☐ [Search Suggestions System](#)
- ☐ [Solution: Search Suggestions System](#)
- ☐ [Replace Words](#)
- ☐ [Solution: Replace Words](#)
- ☐ [Design Add and Search Words Data Structure](#)

- ☐ [Solution: Design Add and Search Words Data Structure](#)
- ☐ [Word Search II](#)
- ☐ [Solution: Word Search II](#)
- ☐ [Top K Frequent Words](#)
- ☐ [Solution: Top K Frequent Words](#)
- ☐ [Longest Common Prefix](#)
- ☐ [Solution: Longest Common Prefix](#)
- ☐ [Index Pairs of a String](#)
- ☐ [Solution: Index Pairs of a String](#)
- ☐ [K-th Smallest in Lexicographical Order](#)
- ☐ [Solution: K-th Smallest in Lexicographical Order](#)
- ☐ [Palindrome Pairs](#)
- ☐ [Solution: Palindrome Pairs](#)
- ☐ [Lexicographical Numbers](#)

24. Hash Maps



Discover the logic behind designing hash maps, solving data structure challenges, and optimal algorithms.

- ☐ [Introduction to Hash Maps](#)
- ☐ [Design HashMap](#)
- ☐ [Solution: Design HashMap](#)
- ☐ [Fraction to Recurring Decimal](#)
- ☐ [Solution: Fraction to Recurring Decimal](#)
- ☐ [Logger Rate Limiter](#)
- ☐ [Solution: Logger Rate Limiter](#)
- ☐ [Next Greater Element I](#)
- ☐ [Solution: Next Greater Element I](#)
- ☐ [Isomorphic Strings](#)
- ☐ [Solution: Isomorphic Strings](#)

- ☐ [Find Duplicate File in System](#)
- ☐ [Solution: Find Duplicate File in System](#)
- ☐ [Dot Product of Two Sparse Vectors](#)
- ☐ [Solution: Dot Product of Two Sparse Vectors](#)
- ☐ [High Five](#)
- ☐ [Solution: High Five](#)
- ☐ [Bulls and Cows](#)
- ☐ [Solution: Bulls and Cows](#)
- ☐ [Custom Sort String](#)
- ☐ [Solution: Custom Sort String](#)
- ☐ [Number of Distinct Islands](#)
- ☐ [Solution: Number of Distinct Islands](#)
- ☐ [Number of Wonderful Substrings](#)
- ☐ [Solution: Number of Wonderful Substrings](#)
- ☐ [Total Appeal of a String](#)
- ☐ [Solution: Total Appeal of a String](#)
- ☐ [Continuous Subarray Sum](#)
- ☐ [Solution: Continuous Subarray Sum](#)
- ☐ [Unique Number of Occurrences](#)
- ☐ [Solution: Unique Number of Occurrences](#)
- ☐ [Longest Happy Prefix](#)
- ☐ [Solution: Longest Happy Prefix](#)
- ☐ [Find Longest Self-Contained Substring](#)
- ☐ [Solution: Find Longest Self-Contained Substring](#)
- ☐ [Longest Palindrome](#)

25. Knowing What to Track



Examine efficient data tracking patterns and solutions for frequency-related problems.

- ☐ [Introduction to Knowing What to Track](#)

- ☐ [Palindrome Permutation](#)
- ☐ [Solution: Palindrome Permutation](#)
- ☐ [Valid Anagram](#)
- ☐ [Solution: Valid Anagram](#)
- ☐ [Design Tic-Tac-Toe](#)
- ☐ [Solution: Design Tic-Tac-Toe](#)
- ☐ [Group Anagrams](#)
- ☐ [Solution: Group Anagrams](#)
- ☐ [Maximum Frequency Stack](#)
- ☐ [Solution: Maximum Frequency Stack](#)
- ☐ [First Unique Character in a String](#)
- ☐ [Solution: First Unique Character in a String](#)
- ☐ [Find All Anagrams in a String](#)
- ☐ [Solution: Find All Anagrams in a String](#)
- ☐ [Longest Palindrome by Concatenating Two-Letter Words](#)
- ☐ [Solution: Longest Palindrome by Concatenating Two-Letter Words](#)
- ☐ [Rank Teams by Votes](#)
- ☐ [Solution: Rank Teams by Votes](#)
- ☐ [Pairs of Songs With Total Durations Divisible by 60](#)
- ☐ [Solution: Pairs of Songs With Total Durations Divisible by 60](#)
- ☐ [Minimum Number of Pushes to Type Word II](#)
- ☐ [Solution: Minimum Number of Pushes to Type Word II](#)
- ☐ [Task Scheduler](#)
- ☐ [Solution: Task Scheduler](#)
- ☐ [Count Anagrams](#)
- ☐ [Solution: Count Anagrams](#)
- ☐ [Divide Array Into Increasing Sequences](#)
- ☐ [Solution: Divide Array Into Increasing Sequences](#)
- ☐ [Ransom Note](#)



26. Union Find

Grasp the fundamentals of the union-find algorithm and its applications in various graph problems.

- ☐ [Introduction to Union Find](#)
- ☐ [Redundant Connection](#)
- ☐ [Solution: Redundant Connection](#)
- ☐ [Number of Islands](#)
- ☐ [Solution: Number of Islands](#)
- ☐ [Most Stones Removed with Same Row or Column](#)
- ☐ [Solution: Most Stones Removed with Same Row or Column](#)
- ☐ [Longest Consecutive Sequence](#)
- ☐ [Solution: Longest Consecutive Sequence](#)
- ☐ [Last Day Where You Can Still Cross](#)
- ☐ [Solution: Last Day Where You Can Still Cross](#)
- ☐ [Regions Cut by Slashes](#)
- ☐ [Solution: Regions Cut by Slashes](#)
- ☐ [Accounts Merge](#)
- ☐ [Solution: Accounts Merge](#)
- ☐ [Minimize Malware Spread](#)
- ☐ [Solution: Minimize Malware Spread](#)
- ☐ [Find if Path Exists in Graph](#)
- ☐ [Solution: Find if Path Exists in Graph](#)
- ☐ [The Skyline Problem](#)
- ☐ [Solution: The Skyline Problem](#)
- ☐ [Evaluate Division](#)



Union Find

Mock Interview

Premium ⓘ



27. Custom Data Structures

Dig into custom data structures for efficient, scalable solutions in complex coding challenges.

- ☐ [Introduction to Custom Data Structures](#)
- ☐ [Snapshot Array](#)
- ☐ [Solution: Snapshot Array](#)
- ☐ [Time Based Key-Value Store](#)
- ☐ [Solution: Time Based Key-Value Store](#)
- ☐ [LRU Cache](#)
- ☐ [Solution: LRU Cache](#)
- ☐ [Insert Delete GetRandom O\(1\)](#)
- ☐ [Solution: Insert Delete GetRandom O\(1\)](#)
- ☐ [Min Stack](#)
- ☐ [Solution: Min Stack](#)
- ☐ [Range Module](#)
- ☐ [Solution: Range Module](#)
- ☐ [Shortest Word Distance II](#)
- ☐ [Solution: Shortest Word Distance II](#)
- ☐ [Design HashSet](#)
- ☐ [Solution: Design HashSet](#)
- ☐ [Max Stack](#)
- ☐ [Solution: Max Stack](#)
- ☐ [Moving Average from Data Stream](#)
- ☐ [Solution: Moving Average from Data Stream](#)
- ☐ [Two Sum III - Data structure design](#)
- ☐ [Solution: Two Sum III - Data structure design](#)
- ☐ [Range Sum Query - Immutable](#)
- ☐ [Solution: Range Sum Query - Immutable](#)
- ☐ [LFU Cache](#)



28. Bitwise Manipulation

Simplify complex topics in bitwise manipulation for efficient problem-solving and optimization.

- ☐ [Introduction to Bitwise Manipulation](#)
- ☐ [Find the Difference](#)
- ☐ [Solution: Find the Difference](#)
- ☐ [Complement of Base 10 Integer](#)
- ☐ [Solution: Complement of Base 10 Integer](#)
- ☐ [Flipping an Image](#)
- ☐ [Solution: Flipping an Image](#)
- ☐ [Single Number](#)
- ☐ [Solution: Single Number](#)
- ☐ [Single Number II](#)
- ☐ [Solution: Single Number II](#)
- ☐ [Encode and Decode Strings](#)
- ☐ [Solution: Encode and Decode Strings](#)
- ☐ [Sum of All Subset XOR Totals](#)
- ☐ [Solution: Sum of All Subset XOR Totals](#)
- ☐ [Find The K-th Lucky Number](#)
- ☐ [Solution: Find The K-th Lucky Number](#)
- ☐ [Minimum Number of K Consecutive Bit Flips](#)
- ☐ [Solution: Minimum Number of K Consecutive Bit Flips](#)
- ☐ [Find the Longest Substring Having Vowels in Even Counts](#)
- ☐ [Solution: Find the Longest Substring Having Vowels in Even Counts](#)
- ☐ [Count Triplets That Can Form Two Arrays of Equal XOR](#)
- ☐ [Solution: Count Triplets That Can Form Two Arrays of Equal XOR](#)
- ☐ [Longest Subarray With Maximum Bitwise AND](#)
- ☐ [Solution: Longest Subarray With Maximum Bitwise AND](#)
- ☐ [Minimum One Bit Operations to Make Integers Zero](#)

- ☐ [Solution: Minimum One Bit Operations to Make Integers Zero](#)
- ☐ [Triples with Bitwise AND Equal To Zero](#)
- ☐ [Solution: Triples with Bitwise AND Equal To Zero](#)
- ☐ [Reverse Bits](#)

29. Math and Geometry ^

Explore mathematical and geometric coding challenges, focusing on efficiency and problem-solving techniques.

- ☐ [Introduction to Math and Geometry](#)
- ☐ [Check If It Is a Straight Line](#)
- ☐ [Solution: Check If It Is a Straight Line](#)
- ☐ [Minimum Cuts to Divide a Circle](#)
- ☐ [Solution: Minimum Cuts to Divide a Circle](#)
- ☐ [Rectangle Overlap](#)
- ☐ [Solution: Rectangle Overlap](#)
- ☐ [Minimum Time Visiting All Points](#)
- ☐ [Solution: Minimum Time Visiting All Points](#)
- ☐ [Reverse Integer](#)
- ☐ [Solution: Reverse Integer](#)
- ☐ [Valid Square](#)
- ☐ [Solution: Valid Square](#)
- ☐ [Rectangle Area](#)
- ☐ [Solution: Rectangle Area](#)
- ☐ [Minimum Area Rectangle](#)
- ☐ [Solution: Minimum Area Rectangle](#)
- ☐ [Maximum Area Rectangle With Point Constraints I](#)
- ☐ [Solution: Maximum Area Rectangle With Point Constraints I](#)
- ☐ [Convex Polygon](#)
- ☐ [Solution: Convex Polygon](#)

- ☐ [Queries on Number of Points Inside a Circle](#)
- ☐ [Solution: Queries on Number of Points Inside a Circle](#)
- ☐ [Max Points on a Line](#)
- ☐ [Solution: Max Points on a Line](#)
- ☐ [Maximum Number of Visible Points](#)
- ☐ [Solution: Maximum Number of Visible Points](#)
- ☐ [Minimize Manhattan Distances](#)
- ☐ [Solution: Minimize Manhattan Distances](#)
- ☐ [Self Crossing](#)
- ☐ [Solution: Self Crossing](#)
- ☐ [Erect the Fence](#)
- ☐ [Solution: Erect the Fence](#)
- ☐ [Minimum Number of Lines to Cover Points](#)
- ☐ [Detonate the Maximum Bombs](#)

30. Challenge Yourself ^

Master the steps to tackle diverse coding problems, enhancing problem-solving skills and strategic thinking.

- ☐ [Challenge Yourself: Introduction](#)
- ☐ [Shortest Bridge](#)
- ☐ [Number of Connected Components in an Undirected Graph](#)
- ☐ [Median of Two Sorted Arrays](#)
- ☐ [Pacific Atlantic Water Flow](#)
- ☐ [Contains Duplicate](#)
- ☐ [Maximum Subarray](#)
- ☐ [Two Sum](#)
- ☐ [Find Minimum in Rotated Sorted Array](#)
- ☐ [Non-overlapping Intervals](#)
- ☐ [Meeting Rooms](#)

- ☐ [Largest Rectangle in Histogram](#)
- ☐ [Subtree of Another Tree](#)
- ☐ [Sort List](#)
- ☐ [Number of 1 Bits](#)
- ☐ [Container with Most Water](#)
- ☐ [Evaluate Reverse Polish Notation](#)
- ☐ [4Sum](#)
- ☐ [Loud and Rich](#)
- ☐ [Product of Array Except Self](#)
- ☐ [Longest Increasing Subsequence](#)
- ☐ [Sum of Two Integers](#)
- ☐ [Majority Element](#)
- ☐ [Unique Paths](#)
- ☐ [Longest Palindromic Substring](#)
- ☐ [Permutations II](#)
- ☐ [Number of Provinces](#)
- ☐ [Linked List Cycle II](#)
- ☐ [Minimum Flips to Make the Binary String Alternate](#)
- ☐ [Lemonade Change](#)
- ☐ [House Robber](#)
- ☐ [Find All Numbers Disappeared in an Array](#)
- ☐ [Find All Duplicates in an Array](#)
- ☐ [Same Tree](#)
- ☐ [Design In-Memory File System](#)
- ☐ [Design File System](#)
- ☐ [Asteroid Collision](#)
- ☐ [Rotting Oranges](#)
- ☐ [Add Binary](#)
- ☐ [Multiply Strings](#)

31. Conclusion



Learn how to use the course's concepts for continued success in coding interviews.



[Where to Go from Here?](#)



Coding Patterns

Mock Interview

Premium ⓘ



Certificate of Completion

Showcase your accomplishment by sharing your certificate of completion.

✅ Developed by MAANG Engineers

Every Educative lesson is designed by our in-house team of ex-MAANG software engineers and PhD computer science educators, and developed in consultation with developers and data scientists working at Meta, Google, and more. Our mission is to get you hands-on with the necessary skills to stay ahead in a constantly changing industry. No video, no fluff. Just interactive, project-based learning with personalized feedback that adapts to your goals and experience.

Trusted by **2.7 million** developers working at companies

NETFLIX



Meta

Google

amazon

airbnb

coinbase

stripe

"Great content, both theoretical and in terms of the selected problems. I do feel much stronger now in tackling various problems and utilizing a diverse set of patterns."

"The flexibility to learn at my own pace, combined with a supportive community, kept me motivated and on track."



Vlad
Learner

Abayomi O
Learner

/
l

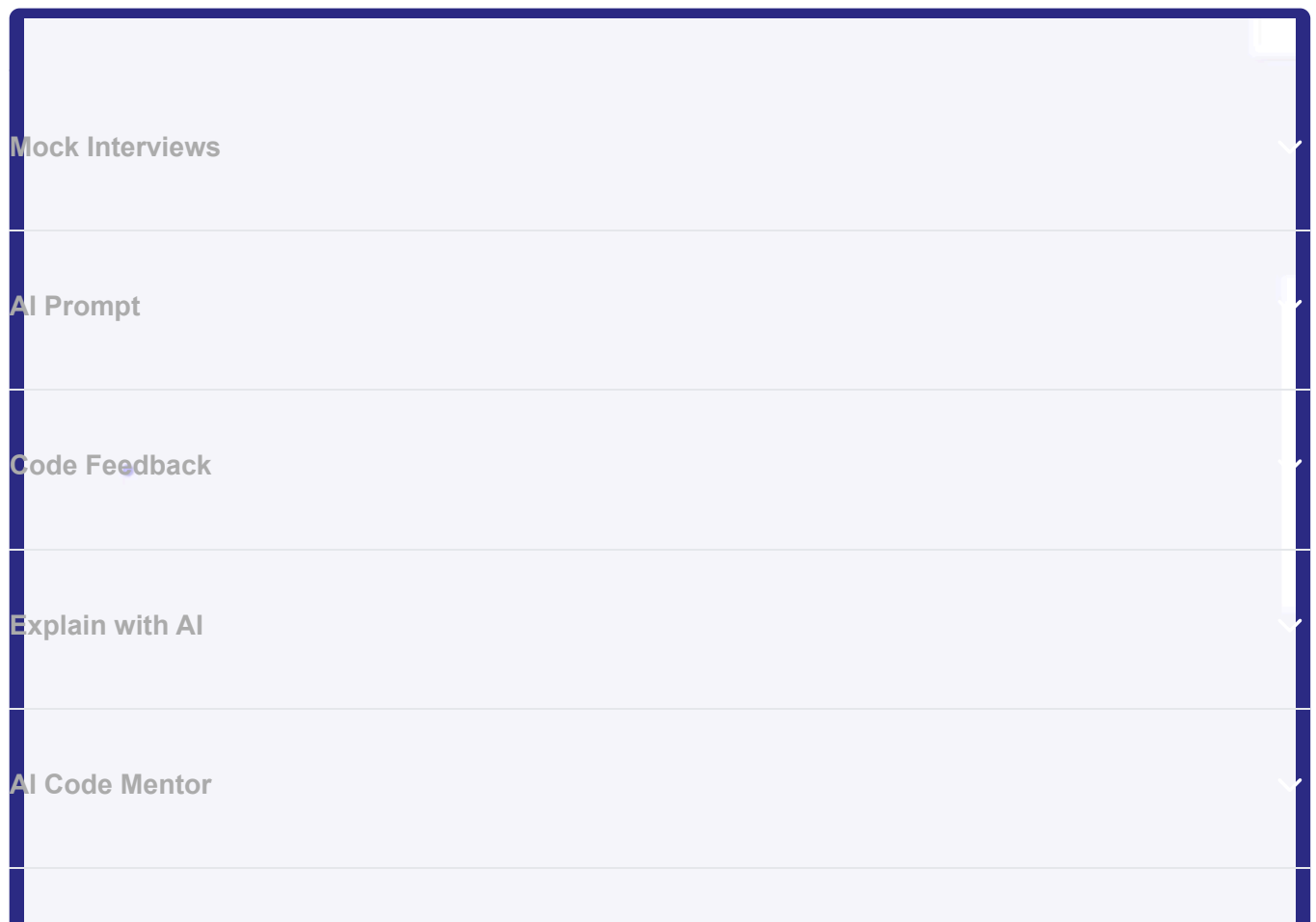
Hands-on Learning Powered by AI

See how Educative uses AI to make your learning more immersive than ever before.

Personalized Interview Prep

Skip the LeetCode grind with a custom roadmap that adapts to your goals. Hands-on practice for Coding Interviews, System Design, and more.

Get Your Free Roadmap



Related Courses and Skill Paths



Skill Path

Java Coding Interview Patterns



Beginner



55 hour



Skill Path

Meta Coding Interview in C++



Beginner



27 hour



Skill Path

Go Coding Interview Patterns



Beginner



55 hour



Skill Path

JavaScript Coding Interview Patterns



Beginner



55 hour



Skill Path

Java Coding Interview



Beginner



181 hour



Skill Path

C++ Coding Interview Prep



Beginner



42 hour



Skill Path

C# Coding Interview



Beginner



125 hour



Skill Path

JavaScript Coding Interview



Intermediate



185 hour



Skill Path

Python Coding Interview Prep



Beginner



42 hour



Course

Coding Interview in Python

 Beginner  15 hour Course

Coding Interview in Elixir

 Beginner  21 hour Skill Path

Meta Coding Interview in Python

 Beginner  27 hour Course

Coding Interview in Go

 Beginner  19 hour Course

Coding Interview in Rust

 Beginner  21 hour Course

Hacking the Coding Interview

 Beginner  40 hour

Free Resources

 Blog[Java Coding Interview Questions](#) Blog[FAANG Coding Interview Questions](#) Blog[JavaScript Coding Interviews](#) Blog[Google Coding Interview](#) Blog[Coding Interview Alternative](#) Blog[Dynamic Coding Interview Problems](#) Blog[Amazon Coding Interview Questions](#) Cheatsheet[Important Coding Interview Problems](#) Blog[Apple Coding Interview Questions](#) Cheatsheet[Techniques for Coding Interviews](#)

 **Blog**

[Cracking the Coding Interview](#)

 **Blog**

[Coding Interview Roadmap](#)

 **Blog**

[C++ Coding Interview Questions](#)

 **Blog**

[Python Algorithms for Coding Interviews](#)

 **Blog**

[Coding Interview FAQs](#)

Frequently Asked Questions

Is Java good for coding interviews?

Yes, Java is excellent for coding interviews. It's a popular and widely used programming language in the industry and effectively supports core data structures and algorithms. Java's strong typing and object-oriented features help write clear, maintainable code, making it a solid choice for technical interviews.

How to crack Java coding interviews?

How can I prepare for a Java interview fast?

What are the coding patterns?

Which coding platform is best for interviews?

Learn in-demand tech skills in half the time

PRODUCTS

- Mock Interview **New**
- Courses
- Cloud Labs
- Skill Paths
- Projects
- Assessments
- Newsletter

TRENDING TOPICS

- Learn to Code
- Tech Interview Prep
- Generative AI
- Data Science
- Machine Learning
- GitHub Students Scholarship
- Early Access Courses
- Blind 75
- Layoffs

PRICING

- For Individuals
- Try for Free
- Gift a Subscription

CONTRIBUTE

- Become an Author
- Become an Affiliate
- Earn Referral Credits

RESOURCES

- Blog
- Cheatsheets
- Webinars
- Answers

ABOUT US

- Our Team
- Careers **Hiring**
- Frequently Asked Questions
- Contact Us
- Press

LEGAL

- Privacy Policy
- Cookie Policy
- Cookie Settings
- Terms of Service

INTERVIEW PREP COURSES

- Grokking the Modern System Design Interview
- Grokking the Product Architecture Design Interview

Business Terms of
Service

Data & Privacy

Data Processing
Agreement

Grokking the Coding
Interview Patterns

Machine Learning
System Design



Copyright ©2025 Educative, Inc. All rights reserved.